

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

Angr 应用示例

实验要求：

根据课本 8.4.3 章节，复现 sym-write 示例的两种 angr 求解方法，并就如何使用 angr 以及怎么解决一些实际问题做一些探讨。

实验过程：

Angr 是一个基于 python 的二进制漏洞分析框架，它将以前多种分析技术集成进来，能够进行动态的符号执行分析（如 KLEE 和 Mayhem），也能够进行多种静态分析。

1. 安装 Angr

因为此前我修改了 vscode 的 py 解释器文件，一直使用的 anaconda 创建的虚拟环境文件夹，这里也不再更改，直接在 anaconda 终端里使用阿里云的镜像下载地址安装 angr：

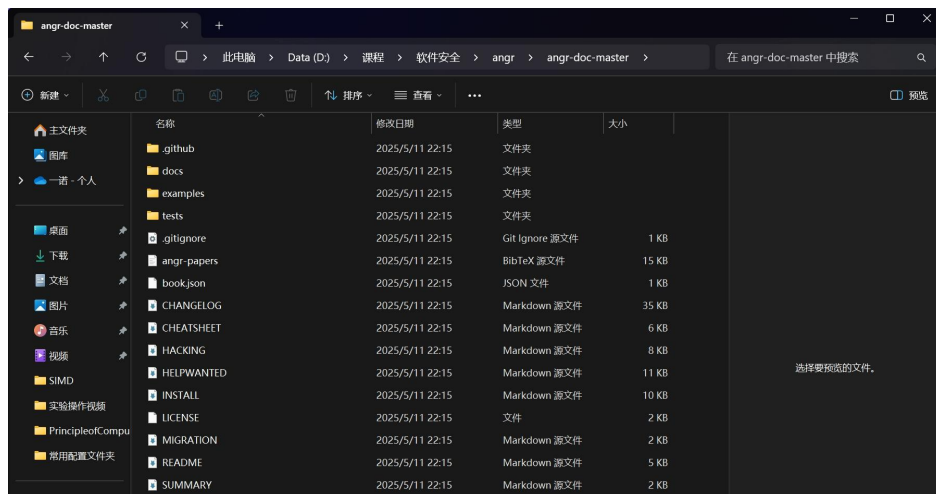
```
(yenv) D:\anaconda3\envs\yenv>pip install angr -i https://pypi.tuna.tsinghua.edu.cn/simple/
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple/
Collecting angr
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/ac/de/8a82e6e95e76cc9c647e41e4e1ed616eab669982b41e3e4cald1dc1d5229/angr-9.2.154-py3-none-win_amd64.whl (4.0 MB)
    4.0/4.0 MB 11.4 MB/s eta 0:00:00
Collecting cxxheaderparser (from angr)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/e4/4c/b1ec3f1555dd35b0559b71eefddb4f57b5d039daf47ee524d0cc8eab2681/cxxheaderparser-1.5.0-py3-none-any.whl (59 kB)
Collecting GitPython (from angr)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/1d/9a/4114a9057db2f1462d5c8f8390ab7383925fe1ac012eaa42402ad65c2963/GitPython-3.1.44-py3-none-any.whl (207 kB)
```

```
Successfully installed GitPython-3.1.44 ailment-9.2.154 angr-9.2.154 archinfo-9.2.154 bitarray-3.4.0 bitstring-4.3.1 capstone-5.0.3 cart-1.2.3 claripy-9.2.154 cle-9.2.154 cxxheaderparser-1.5.0 gitdb-4.0.12 multiplexer-0.9 pefile-2024.8.26 pycryptodome-3.22.0 pydemumble-0.0.1 pydot-4.0.0 pyelftools-0.32 pyformlang-1.0.11 pyvex-9.2.154 smmap-5.0.2 sortedcontainers-2.4.0 unique-log-filter-0.1.0 z3-solver-4.13.0.0
```

安装完成后会有如上图所示字样，此时我们输入 `conda list` 指令查看已安装库，可以看到我们已经将 angr 库安装在了该虚拟环境下。如下图所示：

```
(yenv) D:\anaconda3\envs\yenv>conda list
# packages in environment at D:\anaconda3\envs\yenv:
#
# Name          Version      Build      Channel
absl-py         2.1.0        pypi_0    pypi
ailment         9.2.154      pypi_0    pypi
angr            9.2.154      pypi_0    pypi
annotated-types 0.7.0        pypi_0    pypi
anyio           4.6.2.post1  pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
archinfo        9.2.154      pypi_0    pypi
argon2-cffi     23.1.0       pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
argon2-cffi-bindings 21.2.0      py312h4389bb4_5 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
arrow           1.3.0        pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
```

此时我们打开 vscode，在代码中输入 `import angr`，没有报错，说明 angr 库被成功导入。然后在 Github 下载 angr 的官方文档来获取实验所需样例。解压后如下所示：



2. 复现 sym-write 的两种方法

2.1>方法一

Issue.c 源码:

```
angr-doc-master > examples > sym-write > C issue.c > ...
1  #include <stdio.h>
2
3  char u=0;
4  int main(void)
5  {
6      int i, bits[2]={0,0};
7      for (i=0; i<8; i++) {
8          bits[(u&(1<<i))!=0]++;
9      }
10     if (bits[0]==bits[1]) {
11         printf("you win!");
12     }
13     else {
14         printf("you lose!");
15     }
16     return 0;
17 }
```

Solve.py 源码:

```

angr-doc-master > examples > sym-write > solve.py > main
15 def main():
16     p = angr.Project('angr-doc-master\examples\sym-write\issue', Load_options={"auto_load_libs": False})
17
18     # By default, all symbolic write indices are concretized.
19     state = p.factory.entry_state(add_options={angr.options.SYMBOLIC_WRITE_ADDRESSES})
20
21     u = claripy.BVS("u", 8)
22     state.memory.store(0x804a021, u)
23
24     sm = p.factory.simulation_manager(state)
25
26     def correct(state):
27         try:
28             return b'win' in state.posix.dumps(1)
29         except:
30             return False
31
32     def wrong(state):
33         try:
34             return b'lose' in state.posix.dumps(1)
35         except:
36             return False
37
38     sm.explore(find=correct, avoid=wrong)
39
40     # Alternatively, you can hardcode the addresses.
41     # sm.explore(find=0x80484e3, avoid=0x80484f5)
42
43     return sm.found[0].solver.eval_upto(u, 256)
44
45 def test():
46     good = set()
47     for u in range(256):
48         bits = [0, 0]
49         for i in range(8):
50             bits[u & (1 << i) != 0] += 1
51         if bits[0] == bits[1]:
52             good.add(u)
53
54     res = main()
55     assert set(res) == good
56
57 if __name__ == '__main__':
58     print(repr(main()))

```

在 vscode 中运行，发现得到了 u 的所有结果[51, 57, 60, 240, 75, 139, 78, 197, 23, 142, 90, 209, 29, 154, 212, 99, 163, 102, 108, 166, 172, 105, 169, 114, 53, 120, 178, 184, 71, 135, 77, 83, 89, 141, 147, 153, 92, 202, 86, 156, 150, 101, 165, 43, 226, 232, 46, 113, 116, 177, 45, 180, 58, 198, 15, 201, 195, 85, 204, 210, 30, 149, 27, 39, 216, 106, 225, 170, 228, 54]

，如下图所示：

```

PS D:\课程\软件安全\angr> cd 'd:\课程\软件安全\angr'; & 'd:\anaconda3\envs\python.exe' 'c:\Users\19302\.vscode\extensions\ms-python.debugpy-2025.8.0-win32-x64\bundle\libs\d
cope with this by generating an unconstrained symbolic variable and continuing. You can resolve this by:
WARNING | 2025-05-11 22:29:38,422 | angr.storage.memory_mixins.default_filler_mixin | 1) setting a value to the initial state
WARNING | 2025-05-11 22:29:38,422 | angr.storage.memory_mixins.default_filler_mixin | 2) adding the state option ZERO_FILL_UNCONSTRAINED(MEMORY,REGISTERS), to make unknown regions h
old null
WARNING | 2025-05-11 22:29:38,423 | angr.storage.memory_mixins.default_filler_mixin | 3) adding the state option SYMBOL_FILL_UNCONSTRAINED(MEMORY,REGISTERS), to suppress these messa
ges.
WARNING | 2025-05-11 22:29:38,423 | angr.storage.memory_mixins.default_filler_mixin | Filling register edi with 4 unconstrained bytes referenced from 0x8048521 (__libc_csu_init+0x1 i
n issue (0x8048521))
WARNING | 2025-05-11 22:29:38,425 | angr.storage.memory_mixins.default_filler_mixin | Filling register ebx with 4 unconstrained bytes referenced from 0x8048523 (__libc_csu_init+0x3 i
n issue (0x8048523))
[51, 57, 60, 240, 75, 139, 78, 197, 23, 142, 90, 209, 29, 154, 212, 99, 163, 102, 108, 166, 172, 105, 169, 114, 53, 120, 178, 184, 71, 135, 77, 83, 89, 141, 147, 153, 92, 202, 86, 156
, 150, 101, 165, 43, 226, 232, 46, 113, 116, 177, 45, 180, 58, 198, 15, 201, 195, 85, 204, 210, 30, 149, 27, 39, 216, 106, 225, 170, 228, 54]
PS D:\课程\软件安全\angr>

```

对这种方法进行分析概括：

(1) 新建一个 Angr 工程，并且载入二进制文件。auto_load_libs 设置为 false，将不会自动载入依赖的库，默认情况下设置为 false。

(2) 初始化一个模拟程序状态的 SimState 对象 state，该对象包含了程序的内存、寄存器、文件系统数据、符号信息等等模拟运行时动态变化的数据。

(3) 将要求解的变量符号化，符号化后的变量存在二进制文件的存储区。

(4) 创建模拟管理器进行程序执行管理。初始化的 state 可以经过模拟执行得到一系列的 states，模拟管理器 sm 的作用就是对这些 states 进行管理。

(5) 进行符号执行得到想要的状态，得到想要的状态。上述程序所表达的状态就是，符号执行后，源程序里打印出的字符串里包含 win 字符串，而没有包含 lose 字符串。在这里，状态被定义为两个函数，通过符号执行得到的输出 state.posix.dumps(1) 中是否包含 win 或者 lose 的字符串来完成定义。

(6) 获得到 state 之后，通过 solver 求解器，求解 u 的值。

2.2>方法二

新建一个 solve2.py 来执行第二种方法，源码如下：

```
angr-doc-master > examples > sym-write > solve2.py > ...
1  #!/usr/bin/env python
2  # coding=utf-8
3  import angr
4  import claripy
5
6  def hook_demo(state):
7      state.regs.eax = 0
8
9  p = angr.Project("angr-doc-master\\examples\\sym-write\\issue", load_options={"auto_load_libs": False})
10 # hook函数: addr为待hook的地址
11 # hook为hook的处理函数，在执行到addr时，会执行这个函数，同时把当前的state对象作为参数传递过去
12 # length 为待hook指令的长度，在执行完 hook 函数以后，angr 需要根据 length 来跳过这条指令，执行下一条指令
13 # hook 0x08048485处的指令 (xor eax,eax)，等价于将eax设置为0
14 # hook并不会改变函数逻辑，只是更换实现方式，提升符号执行速度
15 p.hook(addr=0x08048485, hook=hook_demo, length=2)
16 state = p.factory.blank_state(addr=0x0804846B, add_options={"SYMBOLIC_WRITE_ADDRESSES"})
17 u = claripy.BVS("u", 8)
18 state.memory.store(0x0804A021, u)
19 sm = p.factory.simulation_manager(state)
20 sm.explore(find=0x080484DB)
21 st = sm.found[0]
22 |
23 print(repr(st.solver.eval(u)))
```

在 vscode 中运行，结果如下所示：

```
Old null
WARNING | 2025-05-11 22:39:05,071 | angr.storage.memory_mixins.default_filler_mixin | 3) addin
ges.
WARNING | 2025-05-11 22:39:05,071 | angr.storage.memory_mixins.default_filler_mixin | Filling
ssue (0x8048472))
WARNING | 2025-05-11 22:39:05,072 | angr.storage.memory_mixins.default_filler_mixin | Filling
8048475))
226
PS D:\课程\软件安全\angr> |
```

可以看到这种方法与方法一有所差距，只得到了 u 的一个解 226。

这种方法与法一不同，对其进行分析：

(1) 采用了 hook 函数，将 0x08048485 处的长度为 2 的指令通过自定义的 hook_demo 进行替代，功能是一致的，原始 xor eax, eax 和 state.regs.eax = 0 是相同的作用，可以将一些复杂的系统函数调用进行 hook，提升符号执行的性能。

(2) 进行符号执行得到想要的状态变更为 find=0x080484DB。因为源程序 win 和 lose 是互斥的，所以只需给定一个 find 条件即可。

(3) 最后，eval(u) 替代了原来的 eval_upto，将打印一个结果出来。

3. Angr 解决实际问题的探讨

3.1>如何使用

Angr 的使用步骤大致可以概括为如下几步：

1. 安装与环境配置

终端执行 `pip install angr`。推荐使用 Python 3.6+ 和虚拟环境。若遇到依赖问题（如 Z3 或 PyVEX），可通过 `pip install angr-only-z3-custom` 解决。

2. 加载二进制文件

```
import angr
proj = angr.Project("./target_binary", auto_load_libs=False) # 禁用动态
库加载以加速分析
```

3. 设置分析参数

根据具体需求设置初始状态（SimState）以及各种分析选项，如约束条件、路径搜索策略等。

4. 执行分析任务

Angr 提供了各种分析功能，如符号执行、符号执行路径搜索、符号化执行等。以下是一些常见的分析任务：

4.1 符号执行：通过 `project.factory.entry_state()` 创建初始状态，并使用 `project.factory.simulation_manager()` 创建模拟器进行符号执行。

4.2 路径搜索：在符号执行的基础上，使用路径搜索策略来探索程序的不同执行路径。

4.3 约束求解：对于符号执行的结果，使用约束求解器来求解符号变量的具体取值。

5. 处理结果与分析解释

3.2>解决实际问题

Angr 是一个强大的二进制分析框架，结合了符号执行、静态分析和动态分析技术，适用于漏洞挖掘、逆向工程、自动化漏洞利用等场景，在多个问题中均有应用方向：

1. 漏洞挖掘与利用

符号执行：通过动态符号执行（如 KLEE）自动化发现路径约束中的漏洞（如缓冲区溢出、整数溢出等）。

漏洞利用生成：结合约束求解生成 Exploit（如绕过 ASLR、ROP 链构造）。

模糊测试辅助：指导模糊测试（如 AFL）覆盖复杂分支路径。

2. 逆向工程

自动化逆向：识别函数、控制流图（CFG）重建，辅助分析加密算法或混淆代码。

恶意软件分析：静态与动态结合分析恶意代码行为（如 API 调用跟踪、数据流追踪）。

3. 程序验证与合规性检查

属性验证：验证程序是否满足安全属性（如无内存越界访问）。

合规性检测：检查代码是否符合特定规范（如嵌入式设备固件规则）。

4. 二进制补丁分析

差异分析：比较补丁前后二进制文件，定位漏洞修复点。

热补丁生成：通过符号执行生成临时补丁。

5. CTF 竞赛与攻防

自动解题：解决 CTF 中的逆向、Pwn 题目（如自动破解 flag 约束条件）。

攻防模拟：快速验证漏洞攻击链可行性。

6. 系统安全研究

新型分析技术开发：如结合符号执行与抽象解释提升分析效率。

防护机制绕过：研究对抗 CFI、DEP 等防护技术的可能性。

7. 嵌入式与 IoT 安全

固件分析：提取嵌入式设备固件逻辑，检测后门或漏洞。

硬件交互模拟：建模硬件行为以分析驱动或固件代码。

心得体会：

通过实验，对 angr 库有了一定的了解和认识，Angr 是一个基于 python 的二进制漏洞分析框架，它将以前多种分析技术集成进来，能够进行动态的符号执行分析，也能够进行多种静态分析，在多个问题方向中均有应用。

也在一定程度上掌握了其使用方法步骤，同时对 py 库在解决实际问题中的使用有了更深的理解，认识到了工具链使用的方便迅捷。